

Enriching Word Vectors with Subword Information



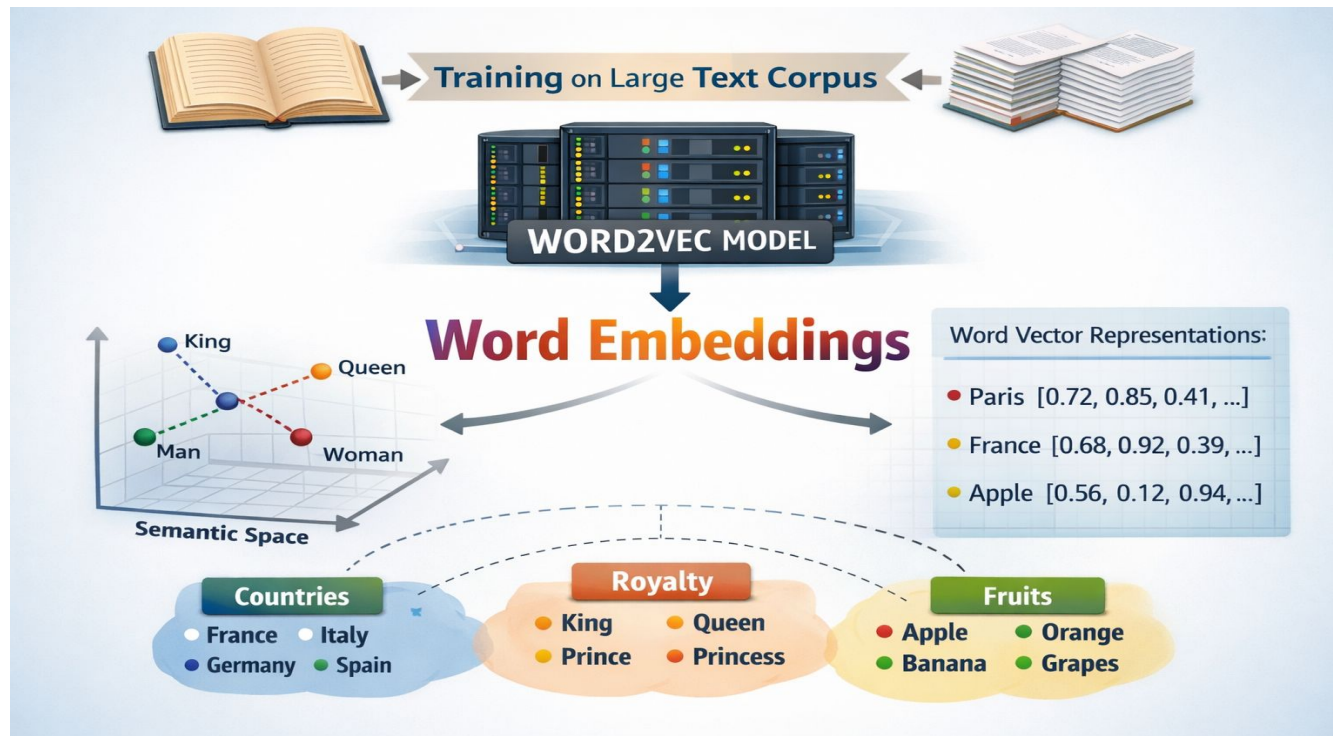
Agenda

- ❑ **Problem Statement**
- ❑ **Motivation**
- ❑ **Background**
- ❑ **Methodology: Subword Model**
- ❑ **Experimentations and Evaluation**
- ❑ **Conclusion**

Problem Statement

Problem Statement

- **Word embeddings** are vector representations of words trained on large corpora e.g., Word2Vec skip-gram and CBOW.



Problem Statement

- Standard models treat each word as an atomic unit with its own vector. This **ignores word internal structure** (morphology).

Standard Word Embeddings

Ignoring Internal Word Structure

Word Vectors as Atomic Units

walk → [0.8, 0.2, -0.6]
walking → [0.5, 0.1, -0.4]
walker → [0.7, -0.3, 0.5]

Separate Vectors

No Morphology Awareness



Treats Each Word as an Independent Entity

Problem Statement

- This is a problem for **rare words**, **morphologically rich languages** (Turkish, Finnish), and out-of-vocabulary (OOV) words.

Finnish 

Noun: koira ('dog')

{ 15 Inflected Forms }

koira	(dog)
koiraki	(with a dog)
koiralle	(for a dog)
koirassasi	(in your dog)
koirista	(about dogs)
koirinakin	(even with dogs)

Spanish / French 

Verb: caminar / marcher ('to walk')

{ 40+ Inflected Forms }

camino	I walk)
caminaremos	(we will walk)
caminabas	(you walked)
camineis	(you all walk)
caminaba	(he was walking)
caminaste	(you walked)

Lots of Word Forms for Nouns and Verbs

Motivation

- Goal: **Leverage subword (character n-gram) information** to enrich representations
-

Background

One Hot Encoding

- Sparsity
- High dimensionality
- No semantic meaning

	1	2	3	4	5	6	7	8
I	1	0	0	0	0	0	0	0
ate	0	1	0	0	0	0	0	0
an	0	0	1	0	0	0	0	0
apple	0	0	0	1	0	0	0	0
and	0	0	0	0	1	0	0	0
played	0	0	0	0	0	1	0	0
the	0	0	0	0	0	0	1	0
piano	0	0	0	0	0	0	0	1

[source: https://towardsdatascience.com/](https://towardsdatascience.com/)

Continuous Bag of words

Input Sentence → "google dream company software engineer"

window_size=3

Independent Features

google

company

o/p

dream

Converting
into one hot
encoding →

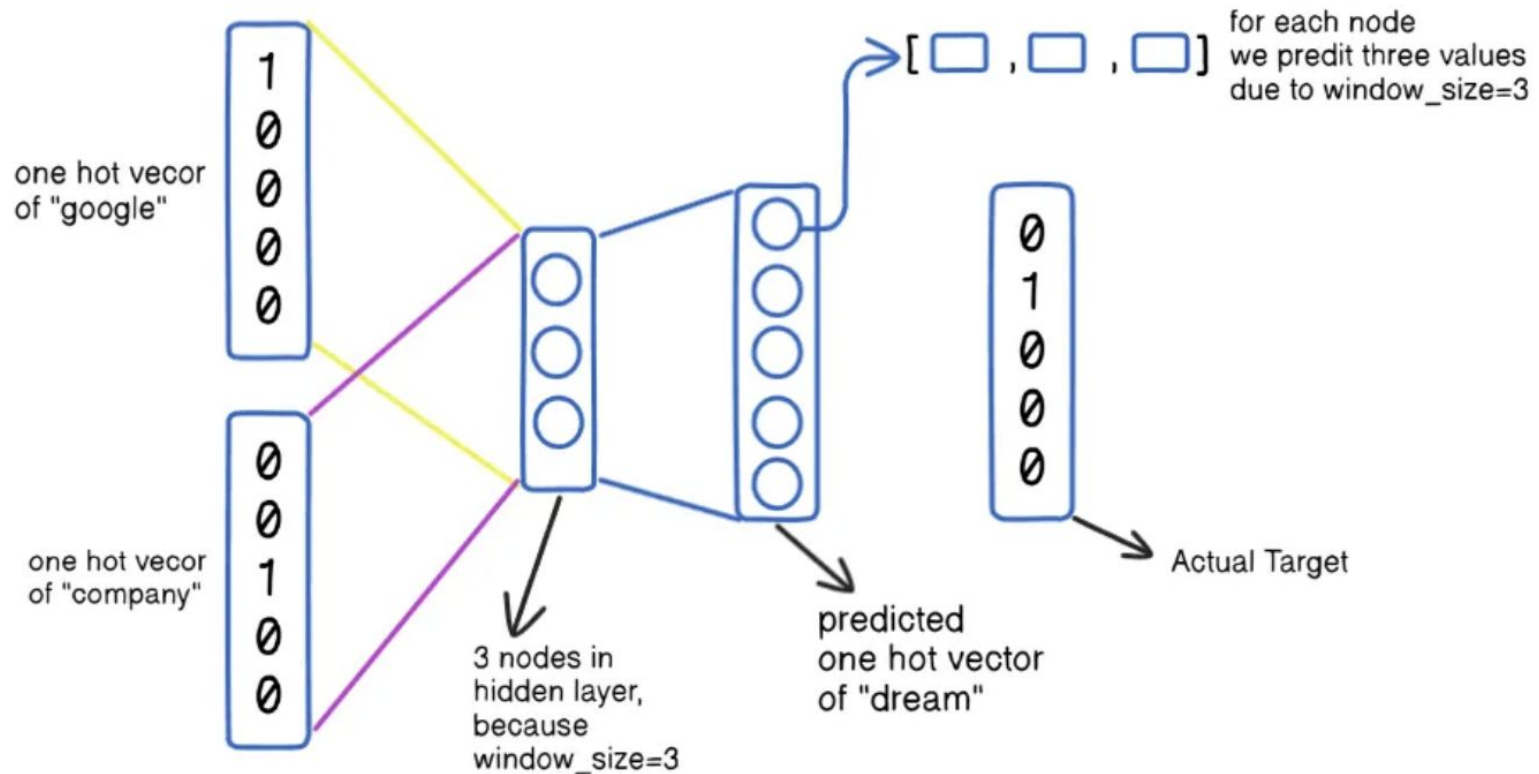
google dream company software engineer

[1 , 0 , 0 , 0 , 0]

[0 , 0 , 1 , 0 , 0]

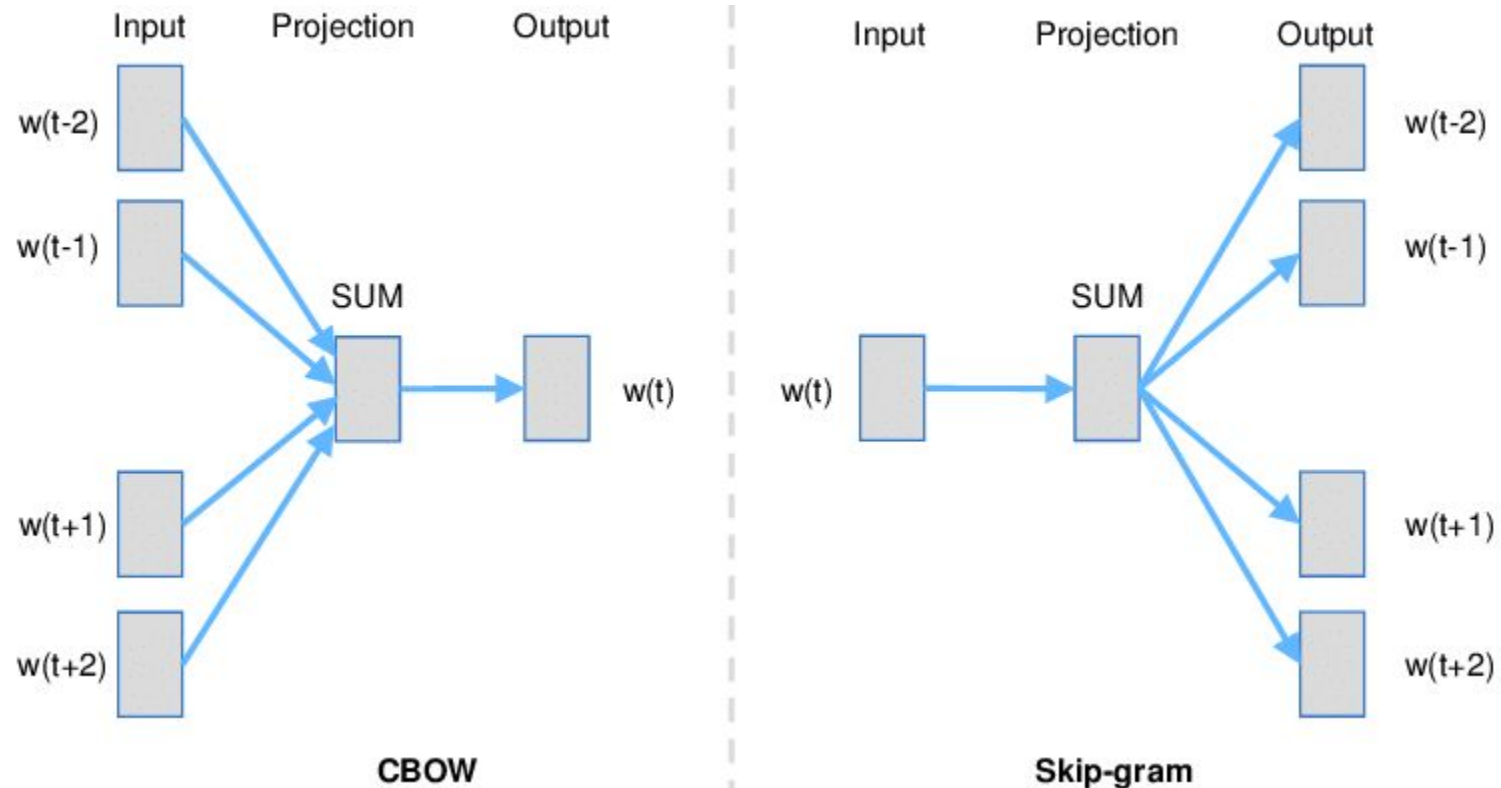
[source: https://medium.com/fraidoonomarzai99](https://medium.com/fraidoonomarzai99)

Continuous Bag of words



source: <https://medium.com/fraidoonomarzai99>

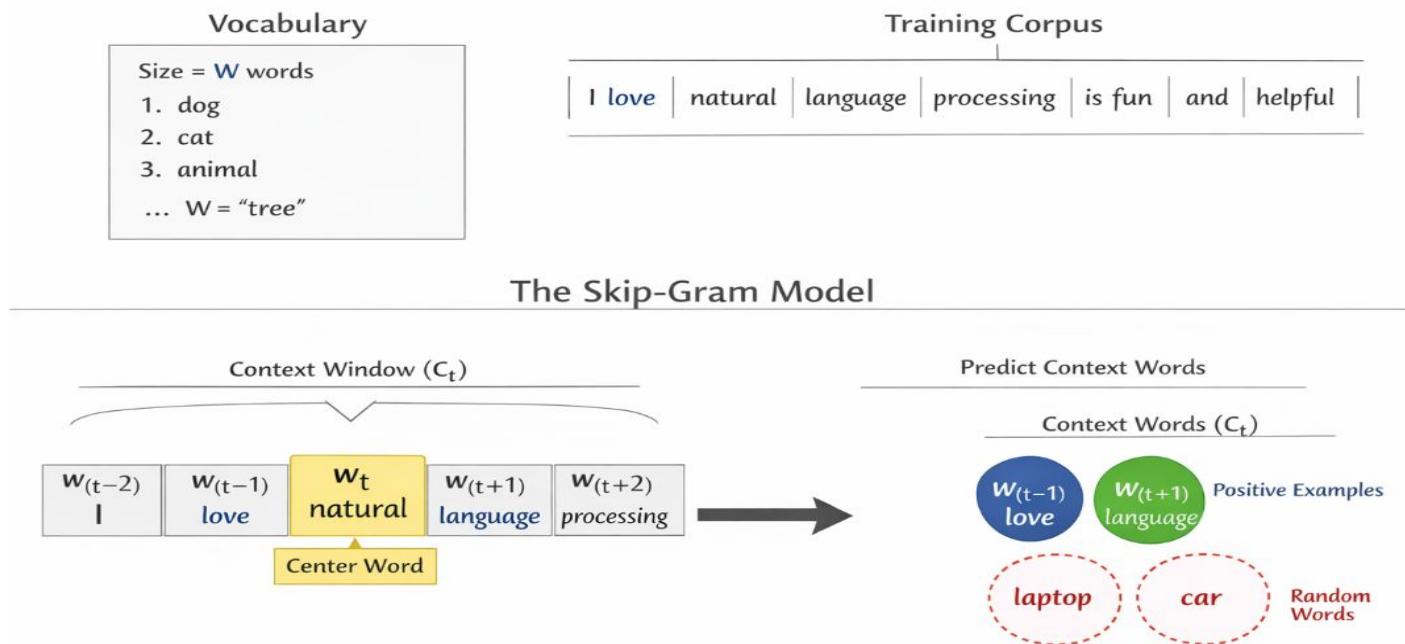
Continuous Bag of words and skip gram



[source: https://www.researchgate.net/](https://www.researchgate.net/)

Skip-gram with Negative Sampling

- **What is the problem we're trying to solve?**
 1. We want to **turn each word into a vector** (a list of numbers) so that **Words that appear in similar contexts have similar vectors**
 2. This idea comes from the **distributional hypothesis**



Skip-gram with Negative Sampling

- We want to **maximize the probability of seeing real context words** given the center word. Mathematically,

$$\sum_{t=1}^T \sum_{c \in \mathcal{C}_t} \log p(w_c | w_t),$$

- Loop over **every word in the corpus**
 - For each word, loop over **each of its context words**
 - Increase the probability that the model assigns to real (center(w_c), context(w_t)) pairs
-

Skip-gram with Negative Sampling

First Idea: Softmax (Too Expensive)

$$p(w_c | w_t) = \frac{e^{s(w_t, w_c)}}{\sum_{j=1}^W e^{s(w_t, j)}}$$

$s(w_t, w_c)$ is a compatibility score
→ it measures how well the center word w_t and context word w_c fit together

To compute softmax probability, we must sum over all words in the vocabulary
→ vocabulary size $W \approx 100,000$ or more

This makes training computationally very expensive
Too slow for large vocabularies

Binary classification instead

$$\log(1 + e^{-s(w_t, w_c)}) + \sum_{n \in \mathcal{N}_{t,c}} \log(1 + e^{s(w_t, n)})$$

Positive Example Loss

Negative Example Loss

Logistic loss denoting as $\ell(x) = \log(1 + e^{-x})$

We can rewrite the objective as:

$$\sum_{t=1}^T \left[\sum_{c \in \mathcal{C}_t} \ell(s(w_t, w_c)) + \sum_{n \in \mathcal{N}_{t,c}} \ell(-s(w_t, n)) \right].$$

Source: <https://arxiv.org/abs/1607.04606>

Skip-gram with Negative Sampling

- Scoring function s between a word w_t and a context word w_c is to use word vectors. Let us define for each word w in the vocabulary two vectors $\mathbf{u}_w$ and $\mathbf{v}_w$ in $\mathbb{R}^d$
- These two vectors are sometimes referred to as input and output vectors in the literature. In particular, we have vectors $\mathbf{u}_{w_t}$ and $\mathbf{v}_{w_c}$, corresponding, respectively, to words w_t and w_c .
- Then the score can be computed as the scalar product between word and context vectors as

$$s(w_t, w_c) = \mathbf{u}_{w_t}^\top \mathbf{v}_{w_c}.$$

*Methodology(Introduced by
Paper)*

Subword

$$s(w_t, w_c) = \mathbf{u}_{w_t}^\top \mathbf{v}_{w_c}.$$

$\mathcal{G}_w \subset \{1, \dots, G\}$ The set of n-gram subwords appearing in word (w)

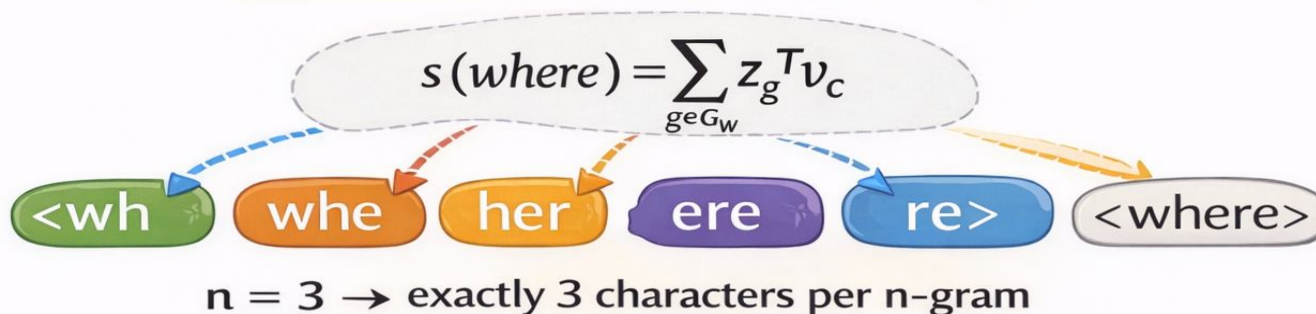
$$s(w, c) = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{v}_c.$$

$$\mathbf{u}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g$$

Subword

Example of Subword Model ($n = 3$)

Word as Bag of Character n-grams



<wh,, whe, her, ere, re> → <where>

Model uses hash function on n-grams with Fowler-Noll-Vo hashing
with Fowler-Noll-Vo hashing

Shares useful n-grams across different words

Model uses hash function on n-grams

$$s(w, c) = \sum_{g \in G_w} z_g^T v_c = \text{with Fowler-Noll-Vo}$$

How subword(n -gram) helps?

	word	n -grams		
DE	autofahrer	fahr	fahrer	auto
	freundeskreis	kreis	kreis>	<freun
	grundwort	wort	wort>	grund
	sprachschule	schul	hschul	sprach
	tageslicht	licht	gesl	tages
EN	anarchy	chy	<anar	narchy
	monarchy	monarc	chy	<monar
	kindness	ness>	ness	kind
	politeness	polite	ness>	eness>
	unlucky	<un	cky>	nlucky
	lifetime	life	<life	time
	starfish	fish	fish>	star
	submarine	marine	sub	marin
FR	transform	trans	<trans	form
	finirais	ais>	nir	fini
	finissent	ent>	finiss	<finis
	finissions	ions>	finiss	sions>

Table 6: Illustration of most important character n -grams for selected words in three languages. For each word, we show the n -grams that, when removed, result in the most different representation.

Source: <https://arxiv.org/abs/1607.04606>

Experimentation and Evaluation

Training Parameter

Training Parameters		Training Datasets	
Parameter	Value	Dataset	Count
Vector Dimension	300	Number of Languages	9
Negatives per Positive	5	Languages Used	
Context Window Size	1 to 5	Arabic, Czech, German, English, Spanish, French, Italian, Romanian, Russian	
Subsampling Threshold	10^{-4}		
Initial Step Size (γ_0)	0.05		
Training Speed	105k words/sec/thread		

Correlation and analogy

		sg	cbow	sisg-	sisg
AR	WS353	51	52	54	55
	GUR350	61	62	64	70
DE	GUR65	78	78	81	81
	ZG222	35	38	41	44
EN	RW	43	43	46	47
	WS353	72	73	71	71
ES	WS353	57	58	58	59
FR	RG65	70	69	75	75
RO	WS353	48	52	51	54
RU	HJ	59	60	60	66

Table 1: Correlation between human judgement and similarity scores on word similarity datasets. We train both our model and the word2vec baseline on normalized Wikipedia dumps. Evaluation datasets contain words that are not part of the training set, so we represent them using null vectors (sisg-). With our model, we also compute vectors for unseen words by summing the n -gram vectors (sisg).

		sg	cbow	sisg
Cs	Semantic	25.7	27.6	27.5
	Syntactic	52.8	55.0	77.8
DE	Semantic	66.5	66.8	62.3
	Syntactic	44.5	45.0	56.4
EN	Semantic	78.5	78.2	77.8
	Syntactic	70.1	69.9	74.9
IT	Semantic	52.3	54.7	52.3
	Syntactic	51.5	51.8	62.7

Table 2: Accuracy of our model and baselines on word analogy tasks for Czech, German, English and Italian. We report results for semantic and syntactic analogies separately.

Source: <https://arxiv.org/abs/1607.04606>

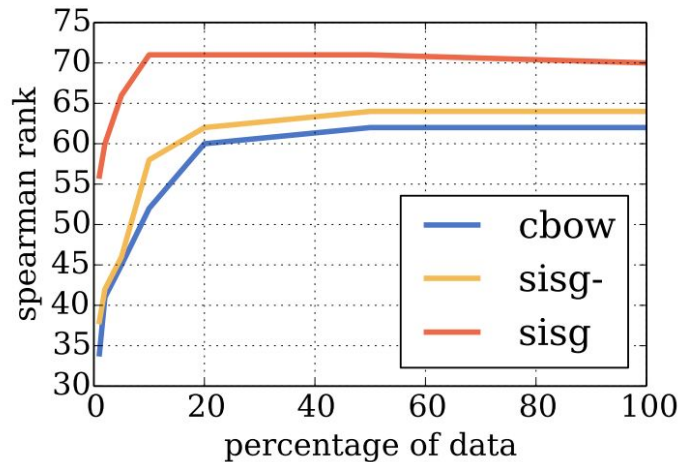
Correlation comparison with different methods

	DE		EN		Es	FR
	GUR350	ZG222	WS353	RW	WS353	RG65
Luong et al. (2013)	-	-	64	34	-	-
Qiu et al. (2014)	-	-	65	33	-	-
Soricut and Och (2015)	64	22	71	42	47	67
sisg	73	43	73	48	54	69
Botha and Blunsom (2014)	56	25	39	30	28	45
sisg	66	34	54	41	49	52

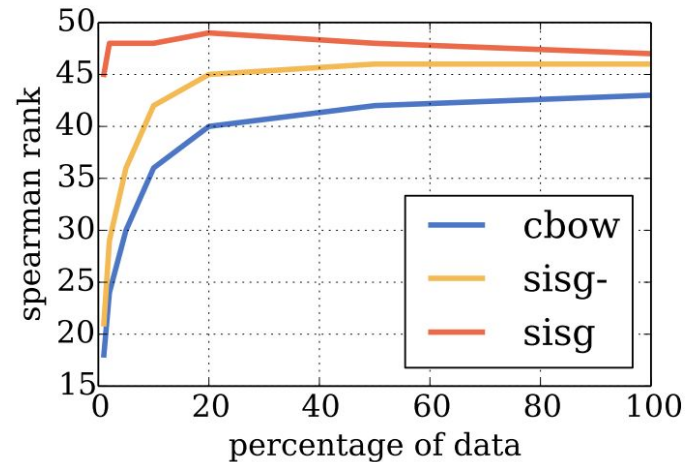
Table 3: Spearman’s rank correlation coefficient between human judgement and model scores for different methods using morphology to learn word representations. We keep all the word pairs of the evaluation set and obtain representations for out-of-vocabulary words with our model by summing the vectors of character n -grams. Our model was trained on the same datasets as the methods we are comparing to (hence the two lines of results for our approach).

Source: <https://arxiv.org/abs/1607.04606>

Training data vs Performance



(a) DE-GUR350



(b) EN-RW

Figure 1: **Influence of size of the training data on performance.** We compute word vectors following the proposed model using datasets of increasing size. In this experiment, we train models on a fraction of the full Wikipedia dump.

Source: <https://arxiv.org/abs/1607.04606>

Effect of the size of the training data

- Proposed model quickly saturates and adding more data does not always improve results.
 - The proposed approach provides very good word vectors even with small training datasets
 - Task-specific data is often scarce, and learning from little data is a big advantage
-

Effect of the size of the n -gram

	2	3	4	5	6
2	57	64	67	69	69
3		65	68	70	70
4			70	70	71
5				69	71
6					70

(a) DE-GUR350

	2	3	4	5	6
2	59	55	56	59	60
3		60	58	60	62
4			62	62	63
5				64	64
6					65

(b) DE Semantic

	2	3	4	5	6
2	45	50	53	54	55
3		51	55	55	56
4			54	56	56
5				56	56
6					54

(c) DE Syntactic

	2	3	4	5	6
2	41	42	46	47	48
3		44	46	48	48
4			47	48	48
5				48	48
6					48

(d) EN-RW

	2	3	4	5	6
2	78	76	75	76	76
3		78	77	78	77
4			79	79	79
5				80	79
6					80

(e) EN Semantic

	2	3	4	5	6
2	70	71	73	74	73
3		72	74	75	74
4			74	75	75
5				74	74
6					72

(f) EN Syntactic

Table 4: Study of the effect of sizes of n -grams considered on performance. We compute word vectors by using character n -grams with n in $\{i, \dots, j\}$ and report performance for various values of i and j . We evaluate this effect on German and English, and represent out-of-vocabulary words using subword information.

Source: <https://arxiv.org/abs/1607.04606>

Effect of the size of the n-gram

- Words are represented using character n-grams instead of whole words, which helps capture word structure like roots and endings.
 - Using n-grams of length **3 to 6** gives a good balance between grammatical information and meaning.
 - Longer n-grams (up to 5 or 6) improve performance, especially for languages with long compound words like German.
-

Effect of the size of the n-gram

- Very short n-grams (such as 2-grams) are less useful because they contain little meaningful information.
 - Since the best n-gram size depends on the language and task, using a **3–6 range** works well across different settings.
-

Language Modeling



Language Model Training Configuration

Category	Parameter	Value / Description
Task	Evaluation task	Language Modeling
Languages	CS, DE, ES, FR, RU	Czech, German, Spanish, French, Russian
Dataset size	Training tokens	~1 million tokens per language
Data setup	Preprocessing & splits	Same as Botha & Blunsom (2014)
Model type		Recurrent Neural Network (RNN)
RNN architecture	LSTM units	650 LSTM units
Regularization	Dropout	0.5
	Weight decay	1×10^{-5}
Optimizer	Optimization method	Adagrad
Learning rate	Learning rate	0.1
Gradient handling	Gradient clipping	Max norm = 1.0
Initialization	Weight initialization	Uniform in $[-0.05, 0.05]$
Training setup	Batch size	20
Baselines	Compared methods	Two baselines (as defined in paper)

Evaluation on Language Modeling task

	Cs	DE	ES	FR	RU
Vocab. size	46k	37k	27k	25k	63k
CLBL	465	296	200	225	304
CANLM	371	239	165	184	261
LSTM	366	222	157	173	262
sg	339	216	150	162	237
sisg	312	206	145	159	206

Table 5: Test perplexity on the language modeling task, for 5 different languages. We compare to two state of the art approaches: CLBL refers to the work of Botha and Blunsom (2014) and CANLM refers to the work of Kim et al. (2016).

Source: <https://arxiv.org/abs/1607.04606>

Conclusion

Conclusion

- The approach incorporates character n-grams into the skip-gram model. Due to its simplicity, the model trains fast and does not require preprocessing or supervision.
 - The model outperforms baselines without subword information as well as methods relying on morphological analysis
-

My opinion

1. Semantic Noise from Subwords

- Subword similarity does **not always correspond to semantic similarity**.
- Words such as **person names, acronyms, or random strings** may share character n-grams but have **no meaningful semantic relationship**.

2. Memory and Computation Trade-offs

- Extending n-grams beyond characters (e.g., **word-level or cross-word n-grams**) is theoretically attractive because it could capture **ordered and contextual information**.
 - However, this is **computationally infeasible in practice** due to rapid vocabulary growth.
 - For example, with **100,000 words**, even simple 2-grams would result in **billions of possible combinations**.
-

My opinion

3. Language Specific Challenges

- The paper analyzes the choice of n-gram length but does not provide a clear guideline for other countries language like chinese.
 - Unlike alphabetic languages, Chinese does not have explicit word boundaries, making n-gram selection more complex.
-

Thank you!!